



Experiment – 1

Name: Simranjeet Kaur

UID: 24BAI70048

Branch: CSE – AIML (CS221)

Section/Group: 24AIT_NTPP-2A

Semester: 5th

Date of Performance: 10/07/2026

Subject: Full Stack-II

Subject Code: 24CSP-337

EXPERIMENT – 1.1

1. **AIM:** To design and develop a dynamic post composer interface supporting multiple platforms with constraint validation.

2. **OBJECTIVE:**

1. To understand multi-platform content handling.
2. To implement real-time validation mechanisms.
3. To design responsive and user-friendly UI components.

3. **SOFTWARE REQUIREMENTS:**

1. Windows/Linux/macOS
2. Node.js (LTS Version)
3. NPM (Node Package Manager)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. React.js
7. Command Prompt / Terminal

4. **THEORY**

React is an open-source JavaScript library developed by Meta (Facebook) for building fast, dynamic, and interactive user interfaces. It follows a component-based architecture, where the user interface is divided into reusable components. This approach improves code reusability, readability, and maintainability.

In modern social media applications, users often create content for multiple platforms such as Twitter, LinkedIn, and Instagram. Therefore, a Post Composer is designed to help users create content while validating platform-specific constraints before publishing. The Post Composer developed in this experiment is implemented using React Functional Components and React Hooks. Whenever the state changes, React automatically updates the user interface without reloading the entire webpage.

User interactions such as typing in the textarea or selecting a platform are handled using event handling (onChange). The application also uses controlled components, where the value of form elements is managed through React state.

The application dynamically calculates the number of characters entered using the length property and compares it with the predefined character limit of the selected platform. Based on this comparison, conditional rendering is used to display appropriate validation messages such as "Ready to Post" or "Character Limit Exceeded."

Overall, this experiment demonstrates the practical implementation of React concepts including components, JSX, state management, event handling, controlled components.



5. IMPLEMENTATION/PROCEDURE

1. Install Node.js and verify the installation using `node -v` and `npm -v`.
2. Create a React project using `npx create-react-app post-composer`.
3. Open the project in Visual Studio Code and run it using `npm start`.
4. Design the Post Composer interface with a heading, platform dropdown, and textarea.
5. Implement React State (`useState`) to manage the selected platform and post content.
6. Add a live character counter using the `length` property.
7. Define platform-specific character limits (Twitter, LinkedIn, and Instagram).
8. Implement real-time validation to display appropriate messages based on the selected platform's character limit.
9. Test the application by entering different post lengths and switching between platforms.
10. Verify that the application displays the correct character count and validation message for each platform.

6. CODE

```
import { useState } from "react";

function App() {
  const [platform, setPlatform] = useState("Twitter");
  const [post, setPost] = useState("");

  const limits = {
    Twitter: 280,
    LinkedIn: 3000,
    Instagram: 2200,
  };

  const limit = limits[platform];
  const isExceeded = post.length > limit;

  return (
    <div
      style={{
        backgroundColor: "#f4f6f9",
        minHeight: "100vh",
        padding: "40px",
        fontFamily: "Arial, sans-serif",
      }}
    >
      <div
        style={{
          width: "650px",
          margin: "auto",
          backgroundColor: "#ffffff",
          padding: "30px",
          borderRadius: "15px",
          boxShadow: "0px 4px 15px rgba(0,0,0,0.2)",
        }}
      >
        <h1
          style={{
            textAlign: "center",
            color: "#2c3e50",
          }}
        />
      </div>
    </div>
  );
}
```



```
        marginBottom: "30px",
    }}
  >
  Social Media Post Composer
</h1>

<label>
  <b>Select Platform</b>
</label>

<br />
<br />

<select
  value={platform}
  onChange={(e) => setPlatform(e.target.value)}
  style={{
    width: "100%",
    padding: "10px",
    borderRadius: "8px",
    fontSize: "16px",
  }}
>
  <option>Twitter</option>
  <option>LinkedIn</option>
  <option>Instagram</option>
</select>

<br />
<br />

<label>
  <b>Write Your Post</b>
</label>

<br />
<br />

<textarea
  rows="6"
  placeholder="Type your post here..."
  value={post}
  onChange={(e) => setPost(e.target.value)}
  style={{
    width: "100%",
    padding: "10px",
    borderRadius: "10px",
    border: "2px solid #ccc",
    fontSize: "16px",
    resize: "none",
  }}
></textarea>

<br />
<br />

<p
  style={{
    fontWeight: "bold",
```

```

        color: isExceeded ? "red" : "#2c3e50",
      }}
    >
    Characters: {post.length} / {limit}
  </p>

  {isExceeded ? (
    <p
      style={{
        color: "red",
        fontWeight: "bold",
      }}
    >
      Character Limit Exceeded!
    </p>
  ) : (
    <p
      style={{
        color: "green",
        fontWeight: "bold",
      }}
    >
      Ready to Post
    </p>
  )}

  <button
    style={{
      backgroundColor: "#007bff",
      color: "white",
      border: "none",
      padding: "12px 20px",
      borderRadius: "8px",
      fontSize: "16px",
      cursor: "pointer",
      marginTop: "10px",
    }}
  >
    Save Draft
  </button>
</div>
</div>
);
}

export default App;

```

7. OUTPUT

The Social Media Post Composer is a user-friendly interface that allows users to create and manage social media posts according to the selected platform's constraints. Users can select a platform such as Twitter, enter their post content in the text area, and view the real-time character count, which is compared with the platform's maximum character limit. The system displays a status message such as “Ready to Post” when the content is within the allowed limit and provides a “Save Draft” button to save the written content for future use. This interface demonstrates the use of platform-specific constraints, interactive form elements, real-time validation, and dynamic user feedback in a React-based application.

Post Composer

Select Platform:

Twitter

Write Your Post:

24BAI70048 Simranjeet Kaur

Characters: 26 / 280

✓ Ready to post

Save Draft

Fig 1.1: Social Media Post Composer User Interface

8. LEARNING OUTCOME

- Understand the basic concepts and purpose of a social media post composer interface.
- Design and develop a user-friendly post composer UI using React.js.
- Implement platform selection features for platforms such as Twitter, LinkedIn, and Instagram.
- Apply platform-specific character limits to ensure content complies with each platform's constraints.
- Use React state management to handle user input and dynamic UI updates.
- Display real-time character counts and warning messages when the content exceeds the allowed limit.

EXPERIMENT – 1.2

1. **AIM:** To implement a draft management system that allows users to save, retrieve, and manage post drafts within the frontend, with optional simulation of backend interactions.

2. OBJECTIVE:

1. To manage draft data using frontend state
2. To implement CRUD operations for draft handling
3. To understand asynchronous workflows in UI applications

3. SOFTWARE REQUIREMENTS:

1. Windows/Linux/macOS
2. Node.js (LTS Version)
3. npm (Node Package Manager)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. React.js
7. Command Prompt / Terminal

4. THEORY

A Draft Management System enables users to temporarily save content before publishing it. It improves user experience by allowing posts to be edited, updated, or deleted at any time. In this experiment, React's `useState()` Hook is used to manage draft data, while `useEffect()` and Local Storage are used to persist drafts even after refreshing the browser. The application also demonstrates CRUD operations, dynamic rendering using `map()`, and array manipulation using `filter()`, providing practical knowledge of frontend state management.

5. IMPLEMENTATION/PROCEDURE

1. Create a React State to store multiple drafts.
2. Implement the Save Draft functionality using `useState()`.
3. Display all saved drafts using the `map()` function.
4. Implement Edit Draft functionality.
5. Implement Delete Draft functionality using `filter()`.
6. Store drafts in the browser using Local Storage.
7. Load saved drafts automatically using `useEffect()`.
8. Test all CRUD operations.

6. CODE

```
import { useState } from "react";

function App() {
  const [platform, setPlatform] = useState("Twitter");
  const [post, setPost] = useState("");
  const [drafts, setDrafts] = useState([]);

  const limits = {
    Twitter: 280,
    LinkedIn: 3000,
    Instagram: 2200,
  };

  const limit = limits[platform];
```



```
const isExceeded = post.length > limit;

const saveDraft = () => {
  if (post.trim() === "") {
    alert("Please write something first!");
    return;
  }

  setDrafts([...drafts, post]);
  setPost("");
};

const editDraft = (index) => {
  setPost(drafts[index]);

  const updatedDrafts = drafts.filter((_, i) => i !== index);
  setDrafts(updatedDrafts);
};

const deleteDraft = (index) => {
  const updatedDrafts = drafts.filter((_, i) => i !== index);
  setDrafts(updatedDrafts);
};

return (
  <div
    style={{
      backgroundColor: "#f4f6f9",
      minHeight: "100vh",
      padding: "40px",
      fontFamily: "Arial",
    }}
  >
    <div
      style={{
        backgroundColor: "white",
        width: "650px",
        margin: "auto",
        padding: "30px",
        borderRadius: "15px",
        boxShadow: "0px 4px 15px rgba(0,0,0,0.2)",
      }}
    >
      <h1
        style={{
          textAlign: "center",
          color: "#2c3e50",
          marginBottom: "30px",
        }}
      >
        Social Media Post Composer
      </h1>

      <label>
        <b>Select Platform</b>
      </label>

      <br />
      <br />
    </div>
  </div>
);
```



```
<select
  value={platform}
  onChange={(e) => setPlatform(e.target.value)}
  style={{
    width: "100%",
    padding: "10px",
    borderRadius: "8px",
    fontSize: "16px",
  }}
>
  <option>Twitter</option>
  <option>LinkedIn</option>
  <option>Instagram</option>
</select>

<br />
<br />

<label>
  <b>Write Your Post</b>
</label>

<br />
<br />

<textarea
  rows="6"
  placeholder="Type your post here..."
  value={post}
  onChange={(e) => setPost(e.target.value)}
  style={{
    width: "100%",
    padding: "10px",
    borderRadius: "10px",
    border: "2px solid #ccc",
    fontSize: "16px",
  }}
></textarea>

<br />
<br />

<p
  style={{
    fontWeight: "bold",
    color: isExceeded ? "red" : "#2c3e50",
  }}
>
  Characters: {post.length} / {limit}
</p>

{isExceeded ? (
  <p
    style={{
      color: "red",
      fontWeight: "bold",
    }}
  >
```



```
Character limit exceeded!
</p>
): (
<p
  style={{
    color: "green",
    fontWeight: "bold",
  }}
>
  Ready to post
</p>
)}

<button
  onClick={saveDraft}
  style={{
    backgroundColor: "#007bff",
    color: "white",
    border: "none",
    padding: "12px 20px",
    borderRadius: "8px",
    cursor: "pointer",
    fontSize: "16px",
    marginTop: "10px",
  }}
>
  Save Draft
</button>

<hr style={{ margin: "30px 0" }} />

<h2> Saved Drafts</h2>

{drafts.length === 0 ? (
  <p>No drafts available.</p>
) : (
  drafts.map((draft, index) => (
    <div
      key={index}
      style={{
        backgroundColor: "#f8f9fa",
        padding: "15px",
        borderRadius: "10px",
        marginBottom: "15px",
        boxShadow: "0px 2px 8px rgba(0,0,0,0.1)",
      }}
    >
      <p>{draft}</p>

      <button
        onClick={() => editDraft(index)}
        style={{
          backgroundColor: "#28a745",
          color: "white",
          border: "none",
          padding: "8px 15px",
          borderRadius: "5px",
          cursor: "pointer",
          marginRight: "10px",
        }}
      >
        Edit
      </button>
    </div>
  ))
)
</p>
)}
</div>
```

```

    }}
  >
  Edit
</button>

<button
  onClick={() => deleteDraft(index)}
  style={{
    backgroundColor: "#dc3545",
    color: "white",
    border: "none",
    padding: "8px 15px",
    borderRadius: "5px",
    cursor: "pointer",
  }}
>
  🗑 Delete
</button>
</div>
))
}}
</div>
</div>
);
}
export default App;

```

7. OUTPUT

A Draft Management System was successfully developed using React.js. The application allows users to save, view, edit, and delete drafts dynamically. Drafts are stored using React State and persisted in Local Storage, ensuring that the data remains available even after refreshing the browser. This experiment demonstrates how React can be used to manage dynamic application data through state management and browser storage. Students gain practical experience with CRUD operations, React Hooks (useState() and useEffect()), array manipulation, Local Storage, and dynamic rendering, which are essential concepts for developing modern frontend applications.

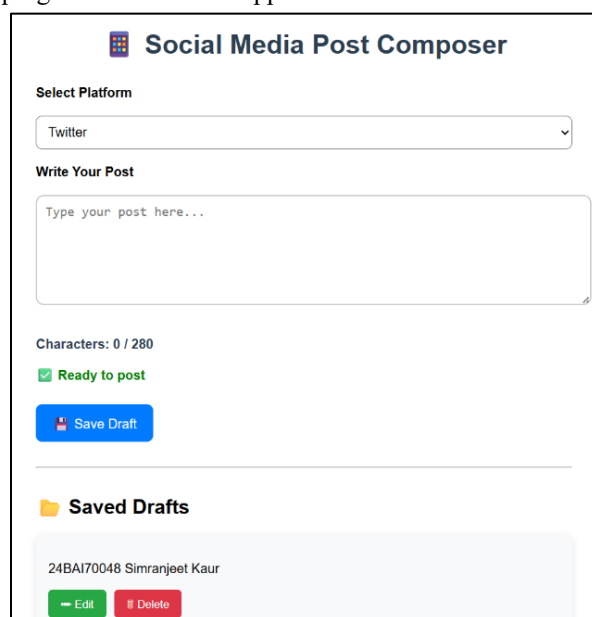


Fig 1.2: Draft Management System Webpage



8. LEARNING OUTCOME

- Understand the concept of Draft Management.
- Implement CRUD operations in React.
- Manage application data using React State.
- Display dynamic data using the map() function.
- Modify arrays using the filter() function.
- Use the useEffect() Hook for lifecycle management.